

# 1. Cel projektu

Celem projektu jest stworzenie kompletnego systemu mobilnego, który umożliwia użytkownikom wykonywanie podstawowych operacji związanych z wymianą walut w sposób szybki, bezpieczny i intuicyjny.

System zapewnia integrację z publicznym API Narodowego Banku Polskiego w celu pobierania aktualnych i archiwalnych kursów walut, a także przechowuje dane użytkowników, ich portfele walutowe i historię transakcji.

System składa się z trzech głównych komponentów:

Aplikacja mobilna (React) – interfejs użytkownika.

Web Service (API) – serwer Ruby on Rails realizujący logikę biznesową.

Baza danych (PostgreSQL) – przechowuje dane użytkowników, kursy, transakcje i portfele.

## 2. Wymagania funkcjonalne

### 1. Użytkownik i autoryzacja

Użytkownik może zarejestrować konto, podając e-mail, hasło i walutę bazową (domyślnie PLN).

Użytkownik może zalogować się do systemu.

System stosuje uwierzytelnianie JWT – każda operacja wymaga ważnego tokena.

Hasła są szyfrowane.

### 2. Zarządzanie portfelem

Użytkownik może zasilić swoje konto (symulowany przelew wirtualny).

Użytkownik może sprawdzić stan posiadanych środków w każdej walucie.

Portfel użytkownika przechowuje salda poszczególnych walut.

### 3. Kursy walut

Aplikacja wyświetla aktualne kursy walut pobrane z API NBP.

System okresowo (co 10 minut) synchronizuje kursy i zapisuje je w bazie.

Użytkownik może przeglądać archiwalne kursy walut z wybranego zakresu dat.

### 4. Wymiana walut

Użytkownik może kupować lub sprzedawać waluty zgodnie z aktualnym kursem.

System automatycznie sprawdza, czy użytkownik ma wystarczające saldo.

System zapisuje każdą transakcję (z datą, kursem, walutami i kwotami).

Po transakcji stan portfela użytkownika zostaje zaktualizowany.

### 5. Historia

Użytkownik może przeglądać historię wszystkich swoich transakcji.

Historia może być filtrowana (np. po dacie, rodzaju transakcji, walucie).

### 6. Dodatkowe funkcje

Alerty kursowe – użytkownik może ustawić powiadomienie, gdy kurs przekroczy określoną wartość.

Wykresy zmian kursów walut (historyczne dane w formie graficznej).

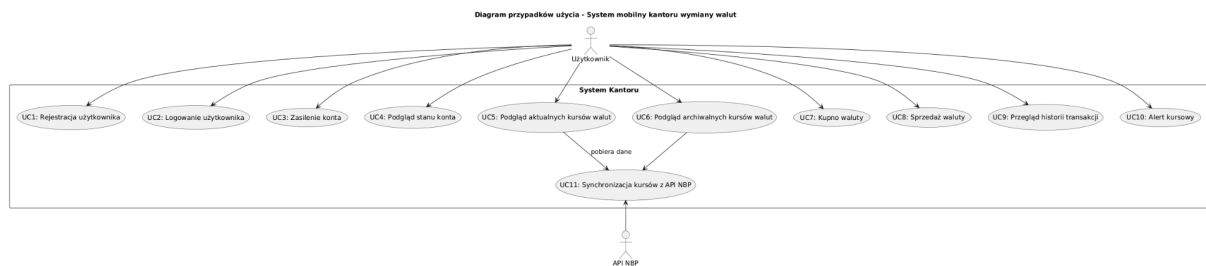
Wersje językowe: polski / angielski.

Tryb offline – możliwość podglądu ostatnio zapisanych kursów bez Internetu.

### 3. Wymagania niefunkcjonalne

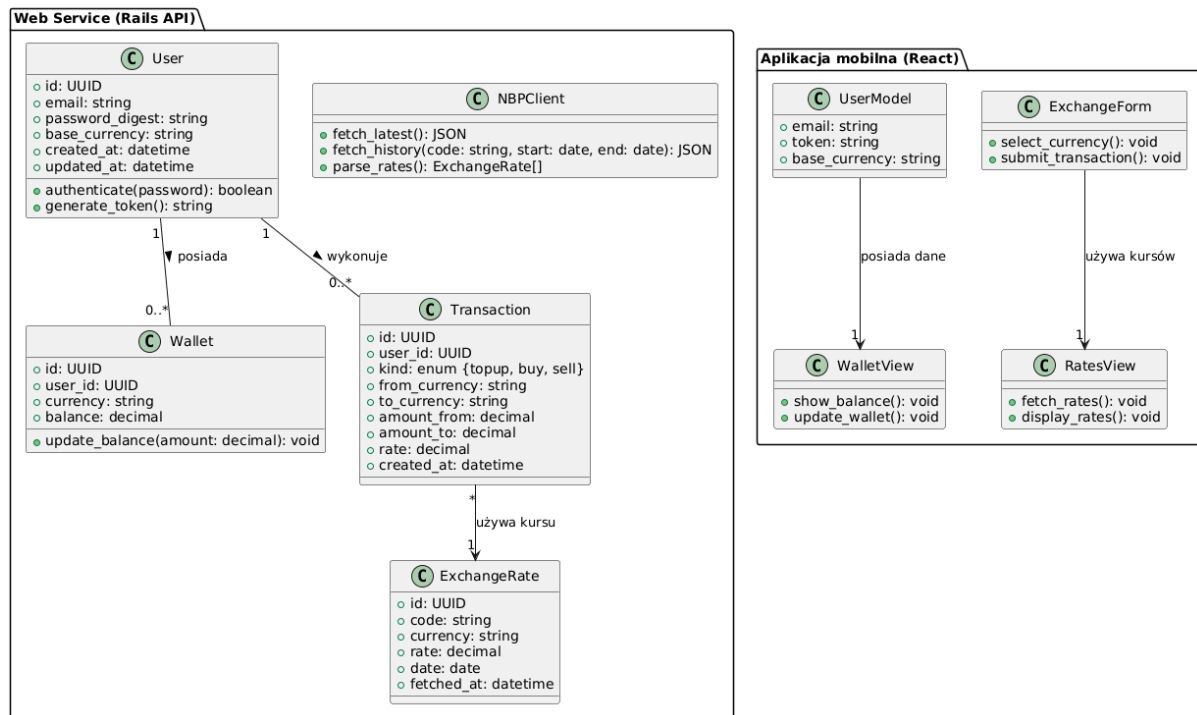
1. **Bezpieczeństwo**  
Hasła szyfrowane, JWT, walidacja danych wejściowych.
2. **Wydajność**  
Kursy NBP buforowane lokalnie w bazie danych (cache 10 min).
3. **Dostępność**  
System konteneryzowany (Docker), łatwe uruchomienie i przenoszenie.
4. **Skalowalność**  
API bezstanowe, możliwe poziome skalowanie.
5. **Użyteczność**  
Responsywny interfejs mobilny, intuicyjne formularze, prosty UX.
6. **Zgodność**  
Komunikacja przez REST, zgodność z API NBP.
7. **Jakość kodu**  
Zastosowanie wzorców MVC, testy jednostkowe i integracyjne.
8. **Niezawodność**  
Obsługa błędów, transakcje w DB, walidacja danych.

### 4. Diagram przypadków użycia



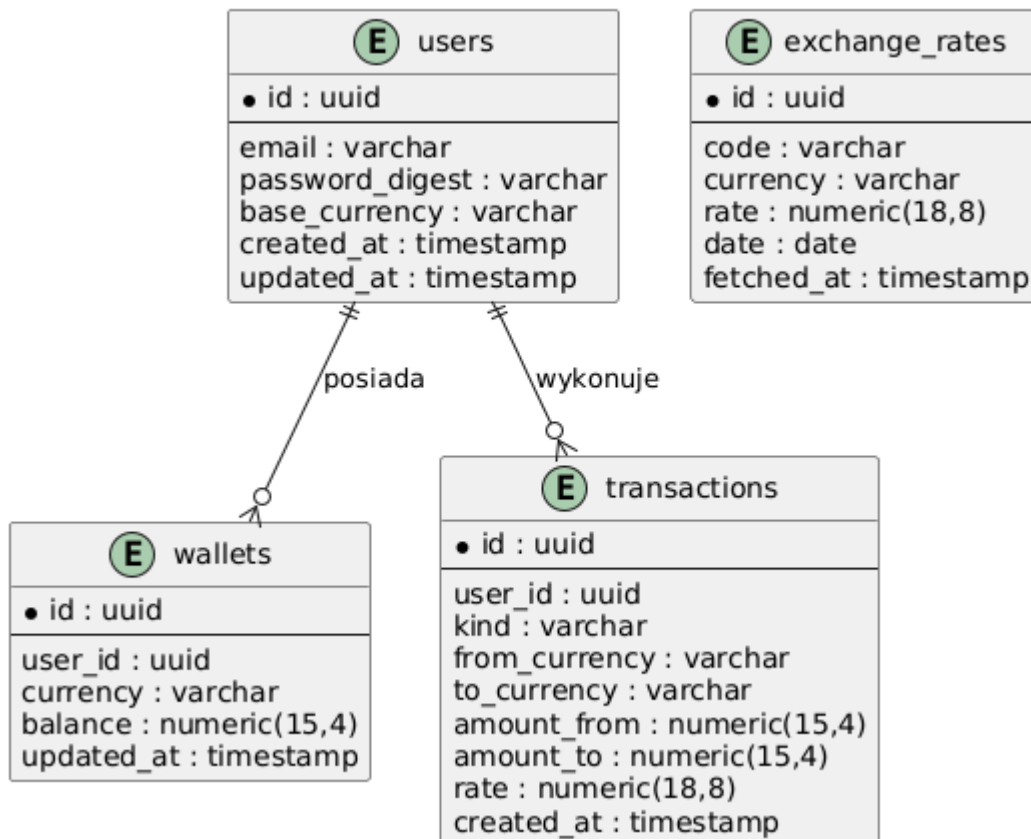
## 4. Diagram UML (klas)

Diagram klas - System Kantoru Wymiany Walut



## 5. Diagram ERD

ERD - System Kantoru Wymiany Walut



## 6. Bezpieczeństwo i Autoryzacja

Bezpieczeństwo danych użytkowników było priorytetem podczas projektowania systemu:

**Uwierzytelnianie:** System korzysta z tokenów JWT. Po zalogowaniu token jest przechowywany w bezpiecznej pamięci urządzenia (AsyncStorage) i dołączany do każdego zapytania w nagłówku Authorization.

**Szyfrowanie haseł:** Hasła nigdy nie są przechowywane w formie jawnej.

**Izolacja danych:** Każdy użytkownik ma dostęp wyłącznie do swoich transakcji i portfeli dzięki mechanizmom filtrowania na poziomie bazy danych w kontrolerach API.

## 7. Interfejs API (Kluczowe Endpointy)

System udostępnia REST-owe API, które pozwala na pełną komunikację aplikacji mobilnej z serwerem:

POST /auth/login – logowanie i generowanie tokena JWT.

POST /auth/register – rejestracja nowego konta z wyborem waluty bazowej.

GET /rates – pobieranie aktualnych kursów walut (z cache lub NBP).

GET /rates/history/:currency – pobieranie danych historycznych do wykresów.

POST /transactions – wykonanie operacji kupna/sprzedaży waluty.

GET /wallets – pobranie aktualnego stanu posiadanych walut użytkownika.

## 8. Mechanizm Synchronizacji Danych

System posiada wbudowany automat (NbpSyncService), który odpowiada za aktualność danych:

Cykliczność: Synchronizacja odbywa się automatycznie (np. co 10 minut lub przy żądaniu użytkownika). Aktualnie trzeba serwis wywołać ale w realnej aplikacji można wykorzystać narzędzie do background jobs takie jak Sidekiq i tam korzystając z crona wywoływać serwis do synchronizacji.

Optymalizacja: Zamiast za każdym razem odpytywać API NBP, system najpierw sprawdza lokalną bazę danych (cache), co drastycznie przyspiesza działanie aplikacji i zmniejsza zużycie danych.

## 9. Kod źródłowy

Github: <https://github.com/PatrykDubinski/aeH-kantor>